

SEGURIDAD OFENSIVA

Escalamiento de Privilegios

Fundamentos teóricos y cuatro casos prácticos
en Linux y Windows

Contenido

1. Introducción

- 1.1. Qué significa "escalar privilegios"
- 1.2. Vertical y horizontal
- 1.3. Dónde encaja en un ataque

2. Conceptos base

- 2.1. Usuarios, grupos y permisos
- 2.2. Las piezas de Linux
- 2.3. Las piezas de Windows

3. Metodología: cómo se busca un camino de escalada

4. Casos prácticos en Linux

- 4.1. Caso 1: Abuso de un binario SUID
- 4.2. Caso 2: Configuración peligrosa de sudo

5. Casos prácticos en Windows

- 5.1. Caso 3: Ruta de servicio sin comillas
- 5.2. Caso 4: AlwaysInstallElevated

6. Herramientas automáticas y qué busca cada una

7. Remediación y hardening (resumen)

1. Introducción

1.1. Qué significa "escalar privilegios"

Escalar privilegios es pasar de un nivel de acceso limitado a uno superior dentro del mismo sistema. En criollo: entrar como un usuario común y terminar teniendo control total de la máquina.

Casi ningún ataque empieza con el máximo poder. Un atacante suele conseguir primero un acceso modesto (una cuenta de usuario sin permisos especiales, una sesión web, un servicio comprometido) y desde ahí busca la forma de convertirse en el usuario más poderoso del sistema. Ese usuario todopoderoso se llama **root** en Linux y **SYSTEM** (o Administrator) en Windows. Con ese nivel se puede leer cualquier archivo, instalar lo que sea, crear usuarios, borrar rastros y, en definitiva, dueñar la máquina.

Lo importante para entender el tema: casi nunca se escala por un "hackeo mágico". Se escala aprovechando **errores de configuración** que dejó un administrador. El sistema funciona como fue programado; el problema es que fue configurado de una forma insegura. Por eso el escalamiento es, en el fondo, un ejercicio de encontrar esos descuidos.

1.2. Vertical y horizontal

- **Escalamiento vertical:** subir de nivel. Por ejemplo, de usuario común a administrador. Es el que más interesa y el que vamos a mostrar en los cuatro casos.
- **Escalamiento horizontal:** moverse hacia otra cuenta del *mismo* nivel para acceder a datos o permisos que esa otra cuenta tiene y la nuestra no. Por ejemplo, robar la sesión de otro usuario común para leer sus archivos.

1.3. Dónde encaja en un ataque

En una intrusión típica el orden es: (1) conseguir un primer acceso, (2) reconocer el sistema por dentro, (3) **escalar privilegios**, (4) usar ese poder para el objetivo final (robar datos, persistir, moverse a otras máquinas). El escalamiento es la bisagra: sin él, el atacante queda encerrado con permisos de invitado.

MARCO DEL TRABAJO

Todo lo que sigue se hace sobre máquinas propias de laboratorio, con fines de aprendizaje. Cada caso incluye su remediación justamente porque el objetivo final de estudiar el ataque es saber defenderse de él.

2. Conceptos base

Antes de los casos conviene tener claras algunas piezas. Sin esto, los comandos parecen magia; con esto, se entiende exactamente por qué funcionan.

2.1. Usuarios, grupos y permisos

Un sistema operativo separa a las personas (y a los programas) en **usuarios**. Cada usuario tiene una identidad y un conjunto de cosas que puede o no hacer. Los usuarios se juntan en **grupos** para asignar permisos de a muchos. Y cada recurso (un archivo, una carpeta, un servicio) tiene reglas sobre quién puede leerlo, modificarlo o ejecutarlo.

La regla de oro que todo administrador debería seguir se llama **principio de mínimo privilegio**: darle a cada usuario y a cada programa solo los permisos que necesita para su tarea, ni uno más. Casi todos los escalamientos existen porque alguien rompió esta regla.

2.2. Las piezas de Linux

Pieza	Qué es y por qué importa
root (UID 0)	El superusuario. UID es el "número de identidad" del usuario; el que tiene UID 0 puede todo. La meta de una escalada en Linux es "volverse root".
Permisos rwx	Cada archivo indica si se puede leer (r), escribir (w) o ejecutar (x), y lo hace por separado para el dueño, el grupo y el resto. Se ven con ls -l .
SUID / SGID	Un permiso especial. Si un programa tiene el bit SUID, al ejecutarlo corre con los permisos de su dueño , no de quien lo lanza. Si el dueño es root, quien lo ejecute obtiene poderes de root <i>mientras dura ese programa</i> . Es la base del Caso 1.
sudo	Un programa que permite a un usuario ejecutar comandos "en nombre de" otro (por defecto, root), según una lista de reglas. Mal configurado, es una puerta directa a root. Es la base del Caso 2.
cron	El planificador de tareas: ejecuta comandos automáticamente cada cierto tiempo, muchas veces como root. Si un atacante puede editar lo que ejecuta una tarea de cron, hereda esos permisos.
PATH	La lista de carpetas donde el sistema busca los programas cuando escribís un comando por su nombre corto. Si un programa privilegiado llama a otro sin ruta completa, y podemos controlar el PATH, podemos hacer que ejecute nuestro programa falso.
capabilities	Una forma moderna de dar poderes puntuales a un programa (por ejemplo, "abrir puertos de red") sin volverlo root entero. Bien usadas mejoran la seguridad; mal asignadas también permiten escalar.

2.3. Las piezas de Windows

Pieza	Qué es y por qué importa
SYSTEM / Administrator	Los niveles máximos. SYSTEM es una cuenta interna aún más poderosa que el propio Administrator: es la que usan los servicios internos del sistema. La meta de una escalada en Windows es llegar a Administrator o, mejor, a SYSTEM.
SID	El identificador único de cada cuenta (el equivalente al UID de Linux). Windows decide qué podés hacer comparando tu SID contra los permisos de cada recurso.
token de acceso	Una especie de "credencial" que Windows le pega a cada proceso e indica en nombre de quién corre y qué privilegios tiene. Varias técnicas avanzadas consisten en robar o suplantar el token de un proceso más privilegiado.
servicio	Un programa que corre de fondo, sin ventana, normalmente arrancando con el sistema y muchas veces como SYSTEM. Cada servicio tiene asociado un ejecutable. Si logramos que un servicio ejecute <i>nuestro</i> programa, corre con los permisos del servicio. Es la base del Caso 3.
UAC	Control de Cuentas de Usuario. Es el cartelito de "¿permitir que esta app haga cambios?". Sirve para que hasta un administrador trabaje con permisos reducidos hasta que confirma. Varias escaladas buscan saltar ese control.
MSI	El formato estándar de los instaladores de Windows (los archivos <code>.msi</code>). El sistema tiene una función para instalar MSI con permisos elevados; si esa función está mal habilitada, cualquiera instala software como SYSTEM. Es la base del Caso 4.

3. Metodología: cómo se busca un camino de escalada

El escalamiento no es adivinanza: sigue un método repetible, y es el mismo para Linux y Windows. El siguiente esquema resume el recorrido completo, desde el acceso inicial hasta el control total.

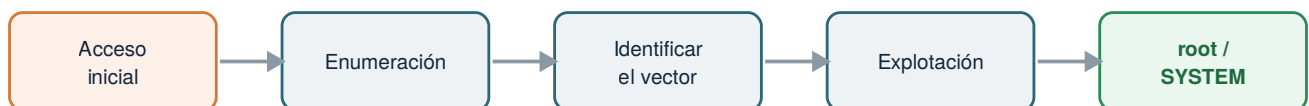


Figura 1. Recorrido de una escalada. La enumeración (paso 2) es la etapa donde se concentra casi todo el trabajo.

Paso 1: Enumeración

Enumerar es **mirar el sistema por dentro y anotar todo lo interesante**: quién soy, a qué grupos pertenezco, qué permisos tengo, qué programas privilegiados hay, qué servicios corren, qué tareas se ejecutan solas, qué archivos puedo escribir, qué versiones de software hay. Es la etapa más importante: el 90% del trabajo es encontrar el descuido, explotarlo suele ser lo rápido. Se puede hacer a mano o con herramientas automáticas (sección 6).

Paso 2: Identificación del vector

Con la información recolectada se busca un **vector**, es decir, una vía concreta de escalada: "este programa tiene SUID y permite abrir una shell", "puedo escribir el ejecutable de este servicio que corre como SYSTEM", etc. Un buen enumerado suele arrojar varios candidatos.

Paso 3: Explotación

Se aprovecha el vector para ejecutar algo con permisos elevados. El objetivo casi siempre es abrir una **shell** (una consola de comandos) que corra como root o SYSTEM, o crear un usuario administrador nuevo.

Paso 4: Comprobación

Se verifica el nuevo nivel. En Linux con el comando `id` (debe mostrar `uid=0(root)`); en Windows con `whoami` (debe mostrar `nt authority\system` o un grupo de administradores). Sin esta comprobación no hay prueba de que la escalada funcionó.

4. Casos prácticos en Linux

4.1. Caso 1: Abuso de un binario SUID

Qué es y por qué ocurre

Como vimos, el bit **SUID** hace que un programa corra con los permisos de su dueño. Existe por buenas razones: por ejemplo, el comando `passwd` (que cambia contraseñas) necesita tocar archivos de sistema, así que es SUID root para que cualquier usuario pueda cambiar su propia clave. El problema aparece cuando el bit SUID está puesto en un programa que **permite abrir una shell o ejecutar comandos arbitrarios**. Si ese programa es SUID root, esa shell corre como root.

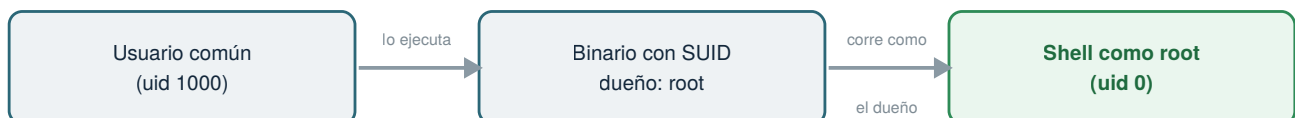


Figura 2. El bit SUID hace que el proceso corra con los permisos del dueño del archivo (root), sin importar quién lo lance.

Muchos programas comunes (editores, buscadores, intérpretes) tienen formas de "escaparse" hacia una consola. Existe un catálogo público llamado **GTFOBins** que documenta, para cada programa, cómo abusarlo si está SUID o si se puede correr por sudo. No es una herramienta de hackeo: es una lista de referencia.

Cómo se detecta (enumeración)

Se listan todos los programas del sistema que tienen el bit SUID:

```
alumno@lab: ~  
  
# Busca en todo el disco archivos con el bit SUID activado.  
# -perm -4000 = tiene SUID ; 2>/dev/null oculta los errores de permiso.  
$ find / -perm -4000 -type f 2>/dev/null
```

Entre la salida aparecen los normales (`passwd`, `sudo`, `mount` ...). Lo que interesa es detectar alguno que **no debería** ser SUID, por ejemplo `find`, `vim`, `python` o un binario a medida. Cada nombre sospechoso se busca en GTFOBins para ver si tiene un escape conocido.

Explotación paso a paso

Supongamos que el enumerado reveló que `find` tiene SUID root (un caso clásico de laboratorio). El comando `find` tiene una opción `-exec` que ejecuta otro programa; la usamos para lanzar una shell:

```
alumno@lab: ~  
  
# 1) Confirmamos quiénes somos: usuario común.  
$ id  
uid=1000(alumno) gid=1000(alumno) groups=1000(alumno)  
  
# 2) Confirmamos que find es SUID root (la 's' en los permisos del dueño).  
$ ls -l /usr/bin/find  
-rwsr-xr-x 1 root root ... /usr/bin/find  
  
# 3) Abusamos de -exec para abrir una shell conservando privilegios (-p).  
$ find . -exec /bin/sh -p \; -quit  
  
# 4) Comprobamos: ahora somos root.  
$ id  
uid=0(root) ...
```

La clave es el `-p` de la shell. Por seguridad, muchas shells (como `bash` y `sh`) al arrancar detectan que "el usuario real y el usuario efectivo no coinciden" y renuncian a los permisos elevados. El `-p` le dice a la shell: no renuncies, conservá los privilegios. Sin ese detalle, la shell se abriría pero como usuario común.

CUÁNDO NO FUNCIONA (PARA MOSTRAR EN LA DEMO)

- a) Si el binario SUID no tiene ningún escape a shell (por ejemplo, un programa a medida que solo suma dos números), no hay forma directa de abusarlo, aunque sea SUID root.
- b) Si abrimos la shell sin el `-p`, arranca pero suelta los privilegios y seguimos siendo el usuario común. Es una falla muy didáctica: se ve que la técnica es correcta pero un detalle la anula.

REMEDIACIÓN / HARDENING

Sacar el bit SUID de todo programa que no lo necesite: `chmod u-s /usr/bin/find`.

Auditar periódicamente la lista de binarios SUID (con el mismo `find` de arriba) y justificar cada uno.

Preferir capabilities antes que SUID cuando un programa solo necesita un poder puntual, para no regalar root entero.

4.2. Caso 2: Configuración peligrosa de sudo

Qué es y por qué ocurre

sudo permite que ciertos usuarios ejecuten ciertos comandos como root, según reglas escritas en el archivo `/etc/sudoers`. Es una herramienta buenísima cuando se usa bien: por ejemplo, dejar que un operador reinicie un servicio puntual sin darle root para todo. El peligro nace de dos descuidos muy comunes:

- Autorizar por sudo un programa que **puede abrir una shell o ejecutar comandos arbitrarios** (los mismos de GTFOBins: editores, intérpretes, etc.). El usuario correría ese programa como root y desde ahí saltaría a una shell root.
- Usar **NOPASSWD**, que permite ejecutar el comando sin siquiera pedir contraseña. Esto no crea el agujero por sí solo, pero lo hace explotable de forma silenciosa e inmediata.

Cómo se detecta (enumeración)

Un solo comando muestra qué nos permite hacer sudo. Una salida peligrosa se ve así:

```
alumno@lab: ~  
# Lista las reglas de sudo que aplican a nuestro usuario.  
$ sudo -l  
User alumno may run the following commands on host:  
  (root) NOPASSWD: /usr/bin/vim
```

Esto dice: "alumno puede ejecutar `vim` como root, sin contraseña". Y `vim` (un editor de texto) tiene un escape a shell conocido. Vector encontrado.

Explotación paso a paso

```
alumno@lab: ~  
  
# 1) Somos usuario común.  
$ id  
uid=1000(alumno) ...  
  
# 2) Ejecutamos vim como root y, desde el editor, usamos '!!' para  
# lanzar una orden del sistema (una shell).  
$ sudo vim -c '!!/bin/sh'  
  
# 3) La shell hereda los permisos de vim: root.  
# id  
uid=0(root) ...
```

La idea de fondo es la misma que en el Caso 1 (aprovechar un programa que puede abrir una consola), pero la vía es distinta: acá el permiso viene de una regla de sudo, no de un bit en el archivo. Por eso cuentan como **dos técnicas diferentes**.

CUÁNDO NO FUNCIONA (PARA MOSTRAR EN LA DEMO)

- a) Si `sudo -l` solo autoriza un programa sin escape a shell (por ejemplo, un script bien escrito que hace una única cosa y no acepta entradas peligrosas), no hay salto posible.
- b) Si la regla *no* tiene NOPASSWD y no conocemos la contraseña del usuario, sudo la pide y quedamos afuera.

REMEDIACIÓN / HARDENING

Mínimo privilegio en sudoers: autorizar solo los comandos estrictamente necesarios, y nunca editores o intérpretes completos.

Evitar NOPASSWD salvo casos muy justificados y acotados.

Para editar archivos con permisos, usar `sudoedit` en lugar de dar sudo sobre el editor: abre una copia con permisos reducidos y evita el escape a shell.

5. Casos prácticos en Windows

5.1. Caso 3: Ruta de servicio sin comillas (Unquoted Service Path)

Qué es y por qué ocurre

Cada **servicio** de Windows tiene guardada la ruta de su ejecutable. Cuando esa ruta contiene espacios y **no está entre comillas**, Windows queda ante una ambigüedad: no sabe dónde termina el nombre del programa y dónde empiezan los argumentos. Entonces parte la ruta en cada espacio y prueba las interpretaciones en orden, agregando `.exe`, hasta que una exista. Ejecuta el **primero que encuentre**.

Ruta guardada del servicio (sin comillas):

```
C:\Program Files\Mi App\servicio.exe
```

Windows prueba de arriba hacia abajo y ejecuta el primero que exista:

1. C:\Program.exe

el atacante coloca acá su .exe con este nombre

2. C:\Program Files\Mi.exe

3. C:\Program Files\Mi App\servicio.exe

el ejecutable legítimo del servicio

Figura 3. Si un usuario puede escribir en `C:\`, planta `Program.exe` y Windows lo ejecuta antes que el servicio real, normalmente como SYSTEM.

Cómo se detecta (enumeración)

Se listan los servicios cuya ruta tiene espacios y no está entre comillas:

cmd.exe

```
:: Lista servicios de arranque automatico, fuera de C:\Windows,  
:: para revisar cuales tienen la ruta sin comillas.  
C:\> wmic service get name,pathname,startmode | findstr /i "auto"
```

Encontrado un candidato, hace falta un segundo requisito: **tener permiso de escritura** en alguna de las carpetas intermedias (por ejemplo `C:\`) y poder reiniciar el servicio, o esperar a que la máquina reinicie. Sin permiso de escritura, el vector no sirve.

Explotación paso a paso

cmd.exe

```
:: 1) Colocamos nuestro .exe malicioso (crea un usuario admin) como:  
:: C:\Program.exe  
  
:: 2) Reiniciamos el servicio (o esperamos un reinicio del equipo).  
C:\> sc stop ServicioVulnerable  
C:\> sc start ServicioVulnerable  
  
:: 3) Windows intenta C:\Program.exe primero y lo ejecuta como SYSTEM.  
:: 4) Verificamos que se creo el usuario administrador.  
C:\> net localgroup administrators
```

CUÁNDO NO FUNCIONA (PARA MOSTRAR EN LA DEMO)

- a) Si no tenemos permiso de escritura en ninguna carpeta intermedia (en un Windows bien mantenido, `C:\` y `C:\Program Files\` están protegidos), no podemos plantar el archivo. Este es el motivo más frecuente por el que falla en la práctica.
- b) Si no podemos reiniciar el servicio ni forzar un reinicio del equipo, el ejecutable queda ahí pero nunca se dispara.

REMEDIACIÓN / HARDENING

Poner comillas en la ruta del servicio para eliminar la ambigüedad, con `sc config` apuntando a la ruta completa entrecomillada.

Restringir permisos de las carpetas para que usuarios comunes no puedan escribir en rutas de instalación ni en `C:\`.

5.2. Caso 4: AlwaysInstallElevated

Qué es y por qué ocurre

AlwaysInstallElevated es una política de Windows que, cuando está activada, hace que **cualquier instalador MSI se ejecute con permisos de SYSTEM**, sin importar quién lo lance. La intención original es legítima: permitir que usuarios comunes instalen software aprobado por la empresa sin molestar al administrador. El problema es que Windows no distingue un MSI "aprobado" de uno malicioso: si la política está encendida, **cualquier** MSI se instala como SYSTEM, incluido uno hecho por el atacante.

La configuración vive en el **registro de Windows** (una base de datos central de opciones del sistema). Para que el agujero exista, el mismo valor tiene que estar en **1** en dos lugares a la vez: uno del equipo (HKLM) y uno del usuario (HKCU). Si solo uno está en 1, no funciona.

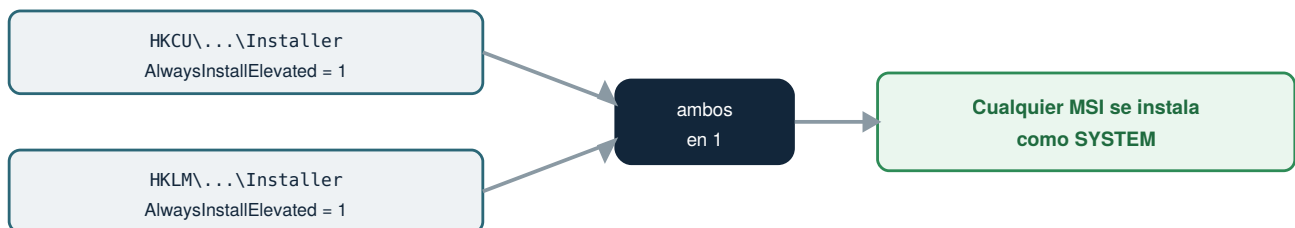


Figura 4. La política solo surte efecto si las dos claves del registro están en 1 a la vez. Con una sola no alcanza.

Cómo se detecta (enumeración)

```
cmd.exe

:: Consultamos el valor en las dos ramas del registro. Ambas deben dar 1.
C:\> reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
C:\> reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

AlwaysInstallElevated    REG_DWORD    0x1
```

Explotación paso a paso

```
cmd.exe

:: 1) Generamos un MSI malicioso (por ejemplo con msfvenom) que cree
::    un usuario admin o abra una shell, y lo copiamos como evil.msi

:: 2) Lo instalamos en silencio. Con la politica activa, corre como SYSTEM.
::    /quiet /qn = sin interfaz ; /i = instalar.
C:\> msixexec /quiet /qn /i C:\ruta\evil.msi

:: 3) Comprobamos.
C:\> whoami
nt authority\system
```

A diferencia del Caso 3, acá no dependemos de un servicio ni de permisos sobre carpetas: el poder viene del propio instalador del sistema. Es una técnica de naturaleza distinta (basada en el instalador MSI y el registro), lo que la vuelve un buen segundo caso de Windows.

CUÁNDO NO FUNCIONA (PARA MOSTRAR EN LA DEMO)

- a) Si solo una de las dos ramas del registro está en 1, la política no se aplica y el MSI se instala con permisos comunes. Sirve para mostrar el requisito de "ambos valores".
- b) Si el antivirus (Windows Defender) detecta el MSI malicioso, lo bloquea antes de que llegue a ejecutarse.

REMEDIACIÓN / HARDENING

Desactivar la política: poner ambos valores del registro en 0 (o eliminarlos). En la práctica, no habilitarla nunca.

Distribuir software por medios controlados (herramientas de despliegue corporativas) en lugar de dejar que usuarios comunes instalen MSI.

6. Herramientas automáticas y qué busca cada una

Enumerar a mano funciona, pero es lento. Existen herramientas que recorren el sistema y marcan solas los posibles vectores. El enunciado pide que, si se usan, se explique **qué buscan**. Acá está el detalle. Son scripts de solo lectura: revisan configuraciones, no atacan por sí mismos.

LinPEAS (Linux)

Es el estándar para Linux. Corre decenas de chequeos y resalta con colores lo más prometedor. Entre lo que busca:

- **Binarios SUID/SGID** inusuales (el vector del Caso 1) y su cruce con GTF0Bins.
- **Permisos de sudo** del usuario actual (el vector del Caso 2).
- **Tareas de cron** que corren como root y archivos editables que ellas ejecutan.

- **Archivos y carpetas escribibles** en lugares sensibles, y carpetas del PATH que podamos controlar.
- **Versión del kernel y del sistema**, para sugerir exploits conocidos si está desactualizado.
- **Contraseñas o claves olvidadas** en archivos de configuración, historiales y variables de entorno.
- **Capabilities** asignadas a programas.

WinPEAS (Windows)

El equivalente para Windows. Busca, entre otras cosas:

- **Servicios con ruta sin comillas** (Caso 3) y servicios cuyo ejecutable o configuración podemos modificar.
- **AlwaysInstallElevated** (Caso 4), consultando las dos ramas del registro.
- **Privilegios especiales del token** del usuario actual, que a veces habilitan técnicas de suplantación.
- **Tareas programadas** y programas de autoarranque modificables.
- **Credenciales guardadas** en el sistema, navegadores y archivos.

PowerUp (Windows, en PowerShell)

Más enfocado y directo. Su comando `Invoke-AllChecks` revisa específicamente las vías de escalada por configuración: servicios modificables, rutas sin comillas, AlwaysInstallElevated y secuestro de librerías (DLL). Es útil porque, además de detectar, muchas veces indica el comando exacto para explotar cada hallazgo.

CÓMO PRESENTARLAS EN LA DEMO

Lo más claro es mostrar primero el hallazgo **a mano** (por ejemplo `sudo -l`) y después correr la herramienta para que confirme lo mismo automáticamente. Así se ve que la herramienta no es magia: aplica los mismos chequeos que hicimos nosotros, pero todos juntos.

7. Remediación y hardening (resumen)

Todas las remediaciones puntuales se apoyan en las mismas ideas de fondo. Vale tenerlas presentes porque es la parte defensiva que el enunciado exige por cada vector.

Vector	Defensa concreta
Binario SUID abusable (Caso 1)	Quitar el bit SUID (<code>chmod u-s</code>), auditar la lista periódicamente, usar capabilities en vez de SUID.
sudo mal configurado (Caso 2)	Autorizar solo comandos sin escape a shell, evitar NOPASSWD, usar <code>sudoedit</code> para editar archivos.
Ruta de servicio sin comillas (Caso 3)	Encomillar la ruta del binario del servicio, restringir escritura en carpetas de instalación y en <code>C:\</code> .
AlwaysInstallElevated (Caso 4)	Dejar la política en 0 en ambas ramas del registro, distribuir software con herramientas corporativas.

Y por encima de todo: **mínimo privilegio** (cada cuenta y cada servicio con lo justo), **actualizaciones al día** (para cerrar vectores por versiones viejas) y **auditoría periódica** con las mismas herramientas de enumeración, esta vez del lado del defensor.